



Amrita School of AI
COIMBATORE CAMPUS

UNIT 1 · SESSIONS 4-7

How fast could this possibly go?

Speedup, efficiency, Amdahl, Gustafson, and the model that tells you when to stop.

Dr. Abhijith Anandakrishnan · Assistant Professor
23AID304 · Odd Semester 2026-27

PART ONE

Speedup and efficiency

$$S(N) = T_1 / T_N$$

$$E(N) = S(N) / N$$

T_1 must be the ****best serial algorithm****. Not your parallel code run with one thread. That hides the overhead in the baseline and flatters the result.

Can speedup exceed N?

- **Usually a mistake**

- Bad serial baseline
- Unoptimised `-O0` build
- Timing the wrong thing

- **Sometimes real**

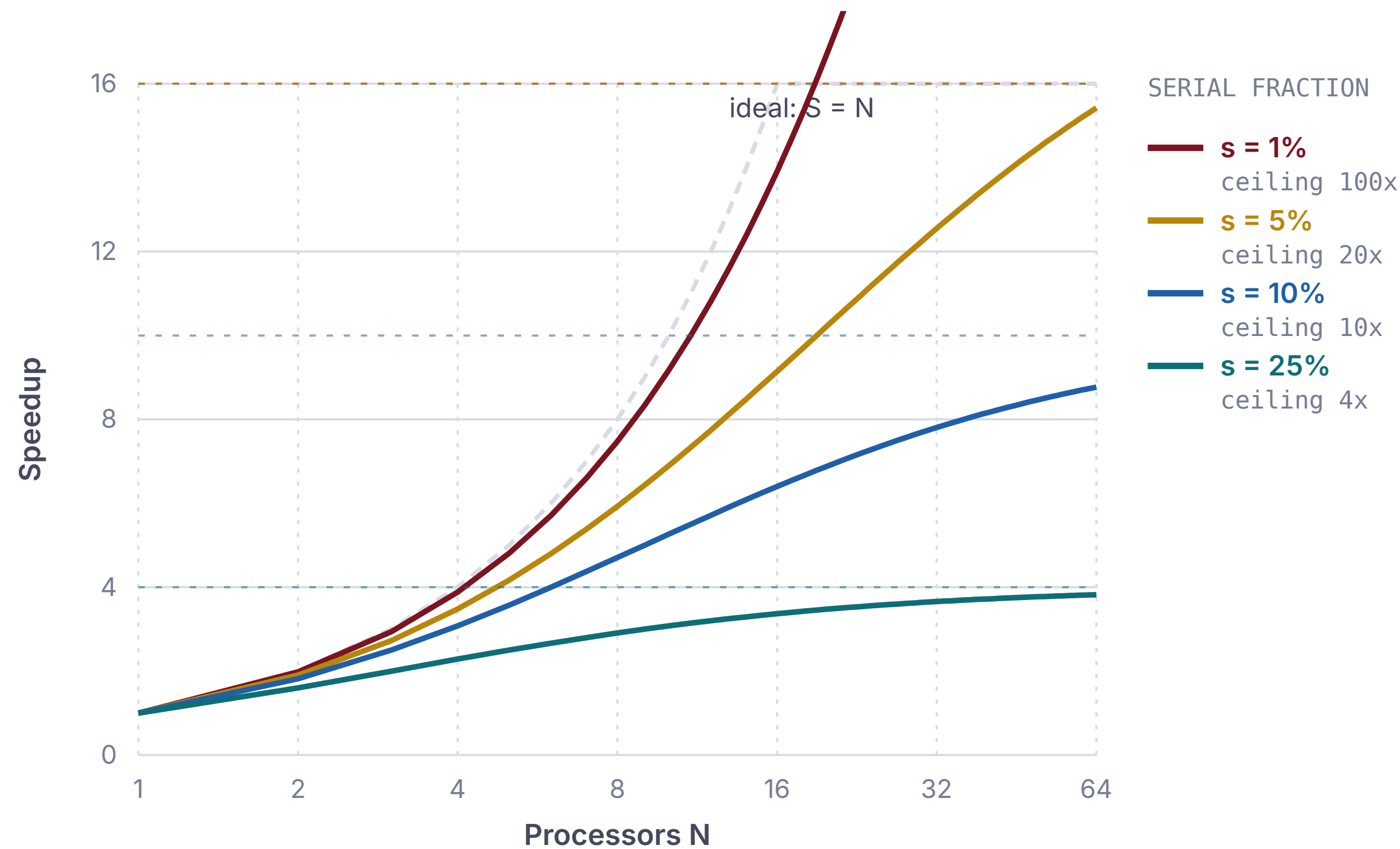
- Each core's slice now fits in cache
- You gained a memory-hierarchy effect on top of parallelism
- **Check the baseline first**

PART TWO

$$S = 1 / (s + (1-s)/N)$$

Amdahl. A fraction s of the work is serial, so speedup can never exceed $1/s$.

5% serial code caps you at 20x. Forever.



What a small serial fraction costs

SERIAL	MAX SPEEDUP
1%	100
5%	20
10%	10
25%	4
50%	2

KARP-FLATT

Run it backwards to find s from a measurement:

$$e = (1/S - 1/N) / (1 - 1/N)$$

THEY DO NOT CONTRADICT EACH OTHER

Amdahl vs Gustafson

- **Amdahl asks**

Fixed problem. More processors.

How much *sooner* does it finish?

→ Hard ceiling at $1/s$

→ Governs **strong scaling**

- **Gustafson asks**

Fixed time budget. More processors.

How much *bigger* a problem fits?

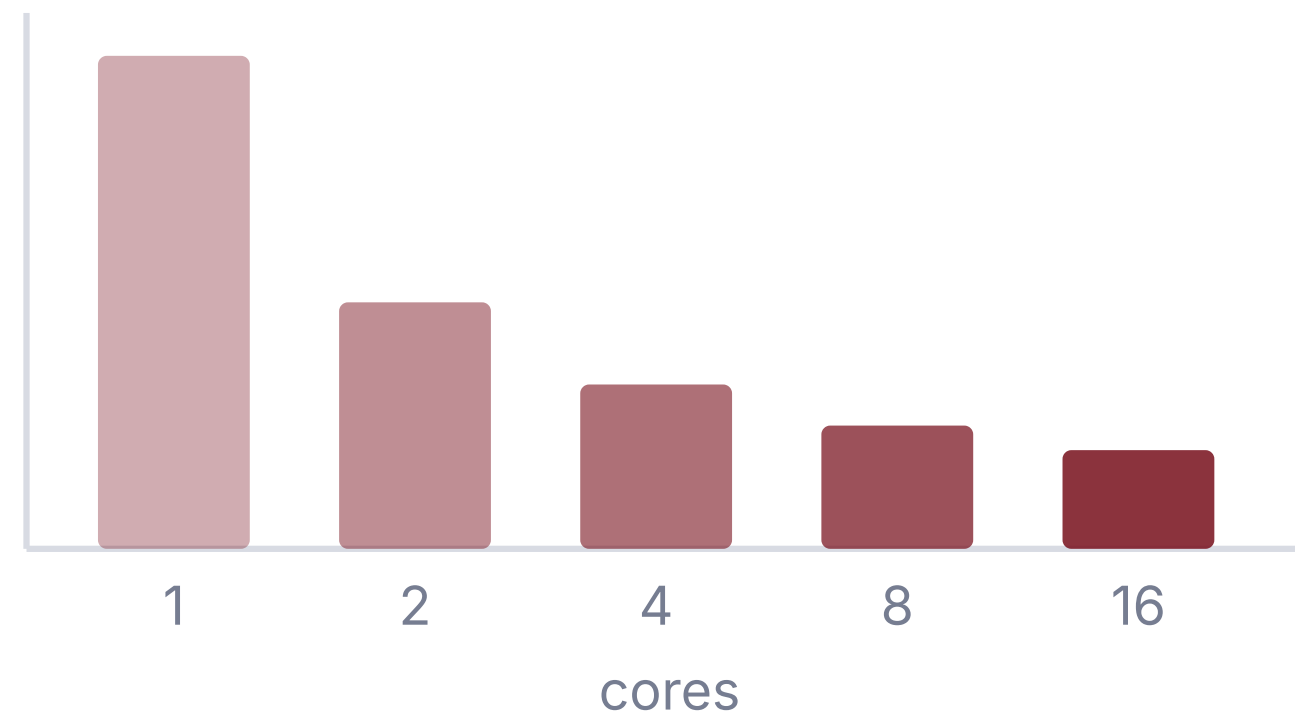
→ Linear in N , no ceiling

→ Governs **weak scaling**

Strong and weak scaling

STRONG SCALING

Problem size fixed. Add cores, finish sooner.

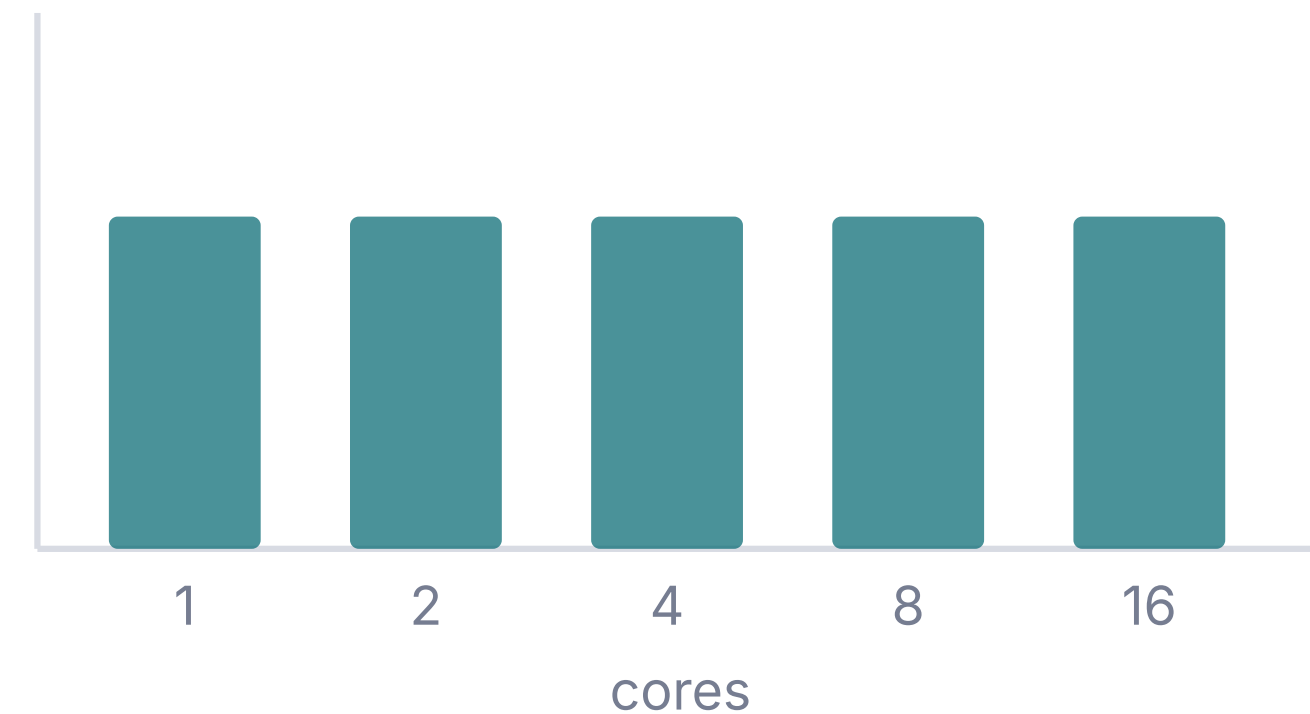


time per core

Work per core shrinks, so communication and serial parts take over. Efficiency falls.

WEAK SCALING

Problem size grows with the cores.



work per core

Work per core is constant, so efficiency can stay high. This is how real HPC scales.

PART THREE

Speedup measures your use of the cores.

It says nothing about your use of the machine.

Arithmetic intensity

$$I = W / Q$$

W flops performed **Q** bytes moved from DRAM

A property of the **algorithm**, not the machine.

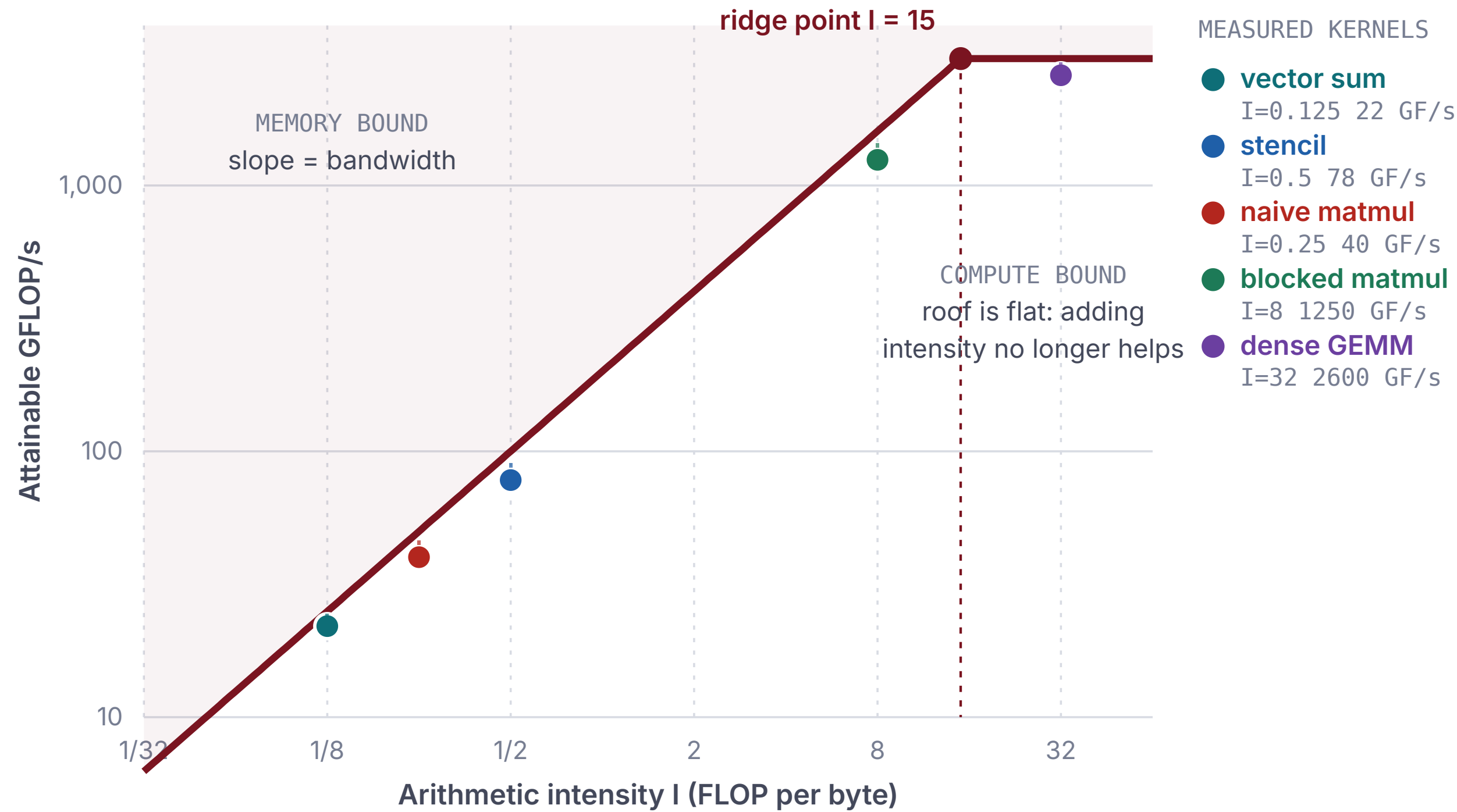
KERNEL	I
vector add	0.04
dot product	0.125
naive matmul	0.25
blocked matmul	~b/4

$$\text{Attainable} = \min(P_{\text{peak}} , \beta \times I)$$

Two ceilings at once: what the arithmetic units can do, and what memory can feed them.

THE ROOFLINE

Where is your kernel, and how much headroom is left?



What the roofline tells you to do

- **Left of the ridge**

Memory bound.

Raise I: block, tile, fuse loops, drop precision.

Optimising the arithmetic will do nothing.

- **Right of the ridge**

Compute bound.

Vectorise, use all cores, check the instruction mix.

If you are at the roof, stop.

"Should I optimise this dot product?"

Machine: $\beta = 200$ GB/s, $P = 3000$ GFLOP/s

Ridge = $3000/200 = 15$ FLOP/byte

Dot product: $l = 2/16 = 0.125$

Ceiling = $200 \times 0.125 = 25$ GFLOP/s

You measure **22 GFLOP/s**. That is 88% of achievable, and 0.8% of peak. **Stop.** Arithmetic is not the constraint.

"3% of peak" can mean perfect.

Always compare against the roofline, never against unqualified peak.

Four questions, in this order

1. What is the ceiling? Compute I , place it on the roofline.
2. How close am I on one core? Fix serial code first.
3. How well does it scale? Run a study, compute E and e .
4. Is more optimisation worth it? Achieved vs attainable.

Next: making it actually happen

OpenMP on the cores, MPI across the nodes.